

Using uv with AWS Lambda

AWS Lambda is a serverless computing service that lets you run code without provisioning or managing servers.

You can use uv with AWS Lambda to manage your Python dependencies, build your deployment package, and deploy your Lambda functions.

!!! tip

Check out the [`uv-aws-lambda-example`](<https://github.com/astral-sh/uv-aws-lambda-example>) project for an example of best practices when using uv to deploy an application to AWS Lambda.

Getting started

To start, assume we have a minimal FastAPI application with the following structure:

```
project
├── pyproject.toml
├── app
│   ├── __init__.py
│   └── main.py
```

Where the `pyproject.toml` contains:

```
[project]
name = "uv-aws-lambda-example"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    # FastAPI is a modern web framework for building APIs with Python.
    "fastapi",
    # Mangum is a library that adapts ASGI applications to AWS Lambda and API
    Gateway.
    "mangum",
]

[dependency-groups]
dev = [
    # In development mode, include the FastAPI development server.
    "fastapi[standard]>=0.115",
]
```

And the `main.py` file contains:

```
import logging

from fastapi import FastAPI
from mangum import Mangum

logger = logging.getLogger()
logger.setLevel(logging.INFO)

app = FastAPI()
handler = Mangum(app)

@app.get("/")
```

```
async def root() -> str:
    return "Hello, world!"
```

We can run this application locally with:

```
$ uv run fastapi dev
```

From there, opening <http://127.0.0.1:8000/> in a web browser will display "Hello, world!"

Deploying a Docker image

To deploy to AWS Lambda, we need to build a container image that includes the application code and dependencies in a single output directory.

We'll follow the principles outlined in the Docker guide (in particular, a multi-stage build) to ensure that the final image is as small and cache-friendly as possible.

In the first stage, we'll populate a single directory with all application code and dependencies. In the second stage, we'll copy this directory over to the final image, omitting the build tools and other unnecessary files.

```
FROM ghcr.io/astral-sh/uv:0.12.7 AS uv

# First, bundle the dependencies into the task root.
FROM public.ecr.aws/lambda/python:3.13 AS builder

# Enable bytecode compilation, to improve cold-start performance.
ENV UV_COMPILE_BYTECODE=1

# Disable installer metadata, to create a deterministic layer.
ENV UV_NO_INSTALLER_METADATA=1

# Enable copy mode to support bind mount caching.
ENV UV_LINK_MODE=copy

# Bundle the dependencies into the Lambda task root via `uv pip install --target`.
#
# Omit any local packages (`--no-emit-workspace`) and development dependencies (`--no-dev`).
# This ensures that the Docker layer cache is only invalidated when the
# `pyproject.toml` or `uv.lock`
# files change, but remains robust to changes in the application code.
RUN --mount=from=uv,source=uv,target=/bin/uv \
    --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
    uv export --frozen --no-emit-workspace --no-dev --no-editable -o requirements.txt
&& \
    uv pip install -r requirements.txt --target "${LAMBDA_TASK_ROOT}"

FROM public.ecr.aws/lambda/python:3.13

# Copy the runtime dependencies from the builder stage.
COPY --from=builder ${LAMBDA_TASK_ROOT} ${LAMBDA_TASK_ROOT}

# Copy the application code.
COPY ./app ${LAMBDA_TASK_ROOT}/app
```

```
# Set the AWS Lambda handler.  
CMD ["app.main.handler"]
```

!!! tip

To deploy to ARM-based AWS Lambda runtimes, replace `public.ecr.aws/lambda/python:3.13` with `public.ecr.aws/lambda/python:3.13-arm64`.

We can build the image with, e.g.:

```
$ uv lock  
$ docker build -t fastapi-app .
```

The core benefits of this Dockerfile structure are as follows:

1. **Minimal image size.** By using a multi-stage build, we can ensure that the final image only includes the application code and dependencies. For example, the uv binary itself is not included in the final image.
2. **Maximal cache reuse.** By installing application dependencies separately from the application code, we can ensure that the Docker layer cache is only invalidated when the dependencies change.

Concretely, rebuilding the image after modifying the application source code can reuse the cached layers, resulting in millisecond builds:

```
=> [internal] load build definition from Dockerfile  
0.0s  
=> => transferring dockerfile: 1.31kB  
0.0s  
=> [internal] load metadata for public.ecr.aws/lambda/python:3.13  
0.3s  
=> [internal] load metadata for ghcr.io/astral-sh/uv:latest  
0.3s  
=> [internal] load .dockerignore  
0.0s  
=> => transferring context: 106B  
0.0s  
=> [uv 1/1] FROM ghcr.io/astral-sh/  
uv:latest@sha256:ea61e006cfec0e8d81fae901ad703e09d2c6cflaa58abcb6507d124b50286f 0.0s  
=> [builder 1/2] FROM public.ecr.aws/lambda/  
python:3.13@sha256:f5b51b377b80bd303fe8055084e2763336ea8920d12955b23ef 0.0s  
=> [internal] load build context  
0.0s  
=> => transferring context: 185B  
0.0s  
=> CACHED [builder 2/2] RUN --mount=from=uv,source=/uv,target=/bin/uv --  
mount=type=cache,target=/root/.cache/u 0.0s  
=> CACHED [stage-2 2/3] COPY --from=builder /var/task /var/task  
0.0s  
=> CACHED [stage-2 3/3] COPY ./app /var/task  
0.0s  
=> exporting to image  
0.0s  
=> => exporting layers  
0.0s  
=> => writing image  
sha256:6f8f9ef715a7cda466b677a9df4046ebbb90c8e88595242ade3b4771f547652d  
0.0
```

After building, we can push the image to Elastic Container Registry (ECR) with, e.g.:

```
$ aws ecr get-login-password --region region | docker login --username AWS --password-stdin aws_account_id.dkr.ecr.region.amazonaws.com
$ docker tag fastapi-app:latest aws_account_id.dkr.ecr.region.amazonaws.com/fastapi-app:latest
$ docker push aws_account_id.dkr.ecr.region.amazonaws.com/fastapi-app:latest
```

Finally, we can deploy the image to AWS Lambda using the AWS Management Console or the AWS CLI, e.g.:

```
$ aws lambda create-function \
  --function-name myFunction \
  --package-type Image \
  --code ImageUri=aws_account_id.dkr.ecr.region.amazonaws.com/fastapi-app:latest \
  --role arn:aws:iam::111122223333:role/my-lambda-role
```

Where the execution role is created via:

```
$ aws iam create-role \
  --role-name my-lambda-role \
  --assume-role-policy-document '{"Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Principal": { "Service": "lambda.amazonaws.com" }, "Action": "sts:AssumeRole" } ]}'
```

Or, update an existing function with:

```
$ aws lambda update-function-code \
  --function-name myFunction \
  --image-uri aws_account_id.dkr.ecr.region.amazonaws.com/fastapi-app:latest \
  --publish
```

To test the Lambda, we can invoke it via the AWS Management Console or the AWS CLI, e.g.:

```
$ aws lambda invoke \
  --function-name myFunction \
  --payload file://event.json \
  --cli-binary-format raw-in-base64-out \
  response.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
```

Where event.json contains the event payload to pass to the Lambda function:

```
{
  "httpMethod": "GET",
  "path": "/",
  "requestContext": {},
  "version": "1.0"
}
```

And response.json contains the response from the Lambda function:

```
{
  "statusCode": 200,
  "headers": {
    "content-length": "14",
    "content-type": "application/json"
  },
}
```

```

    "multiValueHeaders": {},
    "body": "\"Hello, world!\"",
    "isBase64Encoded": false
}

```

For details, see the AWS Lambda documentation.

Workspace support

If a project includes local dependencies (e.g., via Workspaces), those too must be included in the deployment package.

We'll start by extending the above example to include a dependency on a locally-developed library named `library`.

First, we'll create the library itself:

```

$ uv init --lib library
$ uv add ./library

```

Running `uv init` within the project directory will automatically convert project to a workspace and add `library` as a workspace member:

```

[project]
name = "uv-aws-lambda-example"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    # FastAPI is a modern web framework for building APIs with Python.
    "fastapi",
    # A local library.
    "library",
    # Mangum is a library that adapts ASGI applications to AWS Lambda and API
    # Gateway.
    "mangum",
]

[dependency-groups]
dev = [
    # In development mode, include the FastAPI development server.
    "fastapi[standard]",
]

[tool.uv.workspace]
members = ["library"]

[tool.uv.sources]
lib = { workspace = true }

```

By default, `uv init --lib` will create a package that exports a `hello` function. We'll modify the application source code to call that function:

```

import logging

from fastapi import FastAPI
from mangum import Mangum

from library import hello

logger = logging.getLogger()

```

```
logger.setLevel(logging.INFO)
```

```
app = FastAPI()
handler = Mangum(app)
```

```
@app.get("/")
async def root() -> str:
    return hello()
```

We can run the modified application locally with:

```
$ uv run fastapi dev
```

And confirm that opening <http://127.0.0.1:8000/> in a web browser displays, "Hello from library!" (instead of "Hello, World!")

Finally, we'll update the Dockerfile to include the local library in the deployment package:

```
FROM gcr.io/astral-sh/uv:0.12.7 AS uv
```

```
# First, bundle the dependencies into the task root.
```

```
FROM public.ecr.aws/lambda/python:3.13 AS builder
```

```
# Enable bytecode compilation, to improve cold-start performance.
```

```
ENV UV_COMPILE_BYTECODE=1
```

```
# Disable installer metadata, to create a deterministic layer.
```

```
ENV UV_NO_INSTALLER_METADATA=1
```

```
# Enable copy mode to support bind mount caching.
```

```
ENV UV_LINK_MODE=copy
```

```
# Bundle the dependencies into the Lambda task root via `uv pip install --target`.
```

```
#
```

```
# Omit any local packages (`--no-emit-workspace`) and development dependencies (`--no-dev`).
```

```
# This ensures that the Docker layer cache is only invalidated when the  
`pyproject.toml` or `uv.lock`
```

```
# files change, but remains robust to changes in the application code.
```

```
RUN --mount=from=uv,source=uv,target=/bin/uv \
```

```
--mount=type=cache,target=/root/.cache/uv \
```

```
--mount=type=bind,source=uv.lock,target=uv.lock \
```

```
--mount=type=bind,source=pyproject.toml,target=pyproject.toml \
```

```
uv export --frozen --no-emit-workspace --no-dev --no-editable -o requirements.txt  
&& \
```

```
uv pip install -r requirements.txt --target "${LAMBDA_TASK_ROOT}"
```

```
# If you have a workspace, copy it over and install it too.
```

```
#
```

```
# By omitting `--no-emit-workspace`, `library` will be copied into the task root.  
Using a separate
```

```
# `RUN` command ensures that all third-party dependencies are cached separately and  
remain
```

```
# robust to changes in the workspace.
```

```
RUN --mount=from=uv,source=uv,target=/bin/uv \
```

```
--mount=type=cache,target=/root/.cache/uv \
```

```
--mount=type=bind,source=uv.lock,target=uv.lock \
```

```
--mount=type=bind,source=pyproject.toml,target=pyproject.toml \
--mount=type=bind,source=library,target=library \
uv export --frozen --no-dev --no-editable -o requirements.txt && \
uv pip install -r requirements.txt --target "${LAMBDA_TASK_ROOT}"
```

FROM public.ecr.aws/lambda/python:3.13

Copy the runtime dependencies from the builder stage.

COPY --from=builder \${LAMBDA_TASK_ROOT} \${LAMBDA_TASK_ROOT}

Copy the application code.

COPY ./app \${LAMBDA_TASK_ROOT}/app

Set the AWS Lambda handler.

CMD ["app.main.handler"]

!!! tip

To deploy to ARM-based AWS Lambda runtimes, replace `public.ecr.aws/lambda/python:3.13` with `public.ecr.aws/lambda/python:3.13-arm64`.

From there, we can build and deploy the updated image as before.

Deploying a zip archive

AWS Lambda also supports deployment via zip archives. For simple applications, zip archives can be a more straightforward and efficient deployment method than Docker images; however, zip archives are limited to 250 MB in size.

Returning to the FastAPI example, we can bundle the application dependencies into a local directory for AWS Lambda via:

```
$ uv export --frozen --no-dev --no-editable -o requirements.txt
$ uv pip install \
  --no-installer-metadata \
  --no-compile-bytecode \
  --python-platform x86_64-manylinux2014 \
  --python 3.13 \
  --target packages \
  -r requirements.txt
```

!!! tip

To deploy to ARM-based AWS Lambda runtimes, replace `x86_64-manylinux2014` with `aarch64-manylinux2014`.

Following the AWS Lambda documentation, we can then bundle these dependencies into a zip as follows:

```
$ cd packages
$ zip -r ../package.zip .
$ cd ..
```

Finally, we can add the application code to the zip archive:

```
$ zip -r package.zip app
```

We can then deploy the zip archive to AWS Lambda via the AWS Management Console or the AWS CLI, e.g.:

```
$ aws lambda create-function \
  --function-name myFunction \
  --runtime python3.13 \
  --zip-file fileb://package.zip \
  --handler app.main.handler \
  --role arn:aws:iam::111122223333:role/service-role/my-lambda-role
```

Where the execution role is created via:

```
$ aws iam create-role \
  --role-name my-lambda-role \
  --assume-role-policy-document '{"Version": "2012-10-17", "Statement": [{ "Effect":
"Allow", "Principal": {"Service": "lambda.amazonaws.com"}, "Action":
"sts:AssumeRole"}]}'
```

Or, update an existing function with:

```
$ aws lambda update-function-code \
  --function-name myFunction \
  --zip-file fileb://package.zip
```

!!! note

By default, the AWS Management Console assumes a Lambda entrypoint of ``lambda_function.lambda_handler``.

If your application uses a different entrypoint, you'll need to modify it in the AWS Management Console.

For example, the above FastAPI application uses ``app.main.handler``.

To test the Lambda, we can invoke it via the AWS Management Console or the AWS CLI, e.g.:

```
$ aws lambda invoke \
  --function-name myFunction \
  --payload file://event.json \
  --cli-binary-format raw-in-base64-out \
  response.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}
```

Where `event.json` contains the event payload to pass to the Lambda function:

```
{
  "httpMethod": "GET",
  "path": "/",
  "requestContext": {},
  "version": "1.0"
}
```

And `response.json` contains the response from the Lambda function:

```
{
  "statusCode": 200,
  "headers": {
    "content-length": "14",
    "content-type": "application/json"
  },
  "multiValueHeaders": {},
  "body": "\"Hello, world!\""
}
```

```
"isBase64Encoded": false
}
```

Using a Lambda layer

AWS Lambda also supports the deployment of multiple composed Lambda layers when working with zip archives. These layers are conceptually similar to layers in a Docker image, allowing you to separate application code from dependencies.

In particular, we can create a lambda layer for application dependencies and attach it to the Lambda function, separate from the application code itself. This setup can improve cold-start performance for application updates, as the dependencies layer can be reused across deployments.

To create a Lambda layer, we'll follow similar steps, but create two separate zip archives: one for the application code and one for the application dependencies.

First, we'll create the dependency layer. Lambda layers are expected to follow a slightly different structure, so we'll use `--prefix` rather than `--target`:

```
$ uv export --frozen --no-dev --no-editable -o requirements.txt
$ uv pip install \
  --no-installer-metadata \
  --no-compile-bytecode \
  --python-platform x86_64-manylinux2014 \
  --python 3.13 \
  --prefix packages \
  -r requirements.txt
```

We'll then zip the dependencies in adherence with the expected layout for Lambda layers:

```
$ mkdir python
$ cp -r packages/lib python/
$ zip -r layer_content.zip python
```

!!! tip

To generate deterministic zip archives, consider passing the ``-X`` flag to ``zip`` to exclude extended attributes and file system metadata.

And publish the Lambda layer:

```
$ aws lambda publish-layer-version --layer-name dependencies-layer \
  --zip-file fileb://layer_content.zip \
  --compatible-runtimes python3.13 \
  --compatible-architectures "x86_64"
```

We can then create the Lambda function as in the previous example, omitting the dependencies:

```
$ # Zip the application code.
$ zip -r app.zip app

$ # Create the Lambda function.
$ aws lambda create-function \
  --function-name myFunction \
  --runtime python3.13 \
  --zip-file fileb://app.zip \
  --handler app.main.handler \
  --role arn:aws:iam::111122223333:role/service-role/my-lambda-role
```

Finally, we can attach the dependencies layer to the Lambda function, using the ARN returned by the `publish-layer-version` step:

```
$ aws lambda update-function-configuration --function-name myFunction \
  --cli-binary-format raw-in-base64-out \
  --layers "arn:aws:lambda:region:111122223333:layer:dependencies-layer:1"
```

When the application dependencies change, the layer can be updated independently of the application by republishing the layer and updating the Lambda function configuration:

```
$ # Update the dependencies in the layer.
$ aws lambda publish-layer-version --layer-name dependencies-layer \
  --zip-file fileb://layer_content.zip \
  --compatible-runtimes python3.13 \
  --compatible-architectures "x86_64"

$ # Update the Lambda function configuration.
$ aws lambda update-function-configuration --function-name myFunction \
  --cli-binary-format raw-in-base64-out \
  --layers "arn:aws:lambda:region:111122223333:layer:dependencies-layer:2"
```